UNIVERSIDADE DO MINHO

Computer Systems Security (SSC) — MCS, 2026–27

Lab report 3
Bloco 3 — Lab 6, 7 e 8

Setuid, sudo, capabilities e AppArmor

1 Preparação

P1. O real UID é a identidade do processo — o utilizador que o lançou, a quem se imputam os recursos e se dirigem os sinais. O effective UID é a identidade que o kernel usa para decidir o acesso. Numa permissão de ficheiro o kernel compara com o **effective** UID (e com os grupos do processo), nunca com o real.

P2. Quando se executa um ficheiro com o bit setuid, o kernel põe o euid do processo igual ao UID do *dono do ficheiro* e guarda o valor anterior no saved set-user-ID. O processo passa a carregar o privilégio do dono do binário (root, no caso do `backup-tool`); o real UID continua a ser o de quem o lançou. No `passwd` vê-se bem:

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 72056 /usr/bin/passwd
$ ps -o pid,ruser,euser,cmd -C passwd
    PID RUSER    EUSER    CMD
  10527 student  root     passwd
$ grep '^Uid:' /proc/10527/status
Uid:  1000  0  0  0
```

Os quatro valores são real, effective, saved e filesystem. Só o effective mudou: o gid ficou igual, porque o binário é setuid e não setgid.

2 Exercício 1

1a. O mecanismo é o bit setuid, aplicado pelo `execve`. O `passwd` precisa dele porque tem de escrever `/etc/shadow`, que é `0640 root:shadow` e que eu não consigo abrir. Um editor de texto não precisa, porque escreve ficheiros que o utilizador já possui.

Se o `passwd` mantivesse o euid 0 durante toda a execução, passava a ser um processo root que lê input: qualquer falha ao tratar um argumento, uma variável de ambiente ou um ficheiro de configuração dava código a correr como root. O privilégio deve durar só o tempo da operação que o precisa.

3 Exercício 2

```
$ find / -perm -4000 -type f 2>/dev/null | sort
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/fusermount3
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/su
/usr/bin/sudo
/usr/bin/umount
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/local/bin/backup-tool
```

O `find / -perm -2000` deu mais oito setgid: `chage`, `crontab`, `expiry`, `ssh-agent`, `utempter`, `w3mimgdisplay`, `pam_extrausers_chkpwd` e `unix_chkpwd`. São todos do sistema.

2a. Um binário setuid é um design smell porque dá privilégio a quem o executa, e esse privilégio tem de ser justificado um a um — não se assume. Os binários da distribuição já foram escritos para isso e são revistos e actualizados pelo gestor de pacotes. Um binário local como o `backup-tool` não: ninguém o reviu, não tem versão nem origem, e é um `cp` com euid 0, logo aceita dois caminhos quaisquer.

4 Exercício 3

```
$ /usr/local/bin/backup-tool /etc/hostname /tmp/hn && cat /tmp/hn
ssc-lab

$ cat /etc/shadow
cat: /etc/shadow: Permission denied

$ /usr/local/bin/backup-tool /etc/shadow /tmp/leak; echo "rc=$?"
rc=0
$ head -2 /tmp/leak
root:*:20679:0:99999:7:::
daemon:*:20679:0:99999:7:::

$ printf 'planted-by-svc-report\n' > /tmp/payload
$ /usr/local/bin/backup-tool /tmp/payload /root/pwned.txt; echo "rc=$?"
rc=0
$ sudo cat /root/pwned.txt
planted-by-svc-report
```

3a. Usei a via de **leitura**: não consigo ler `/etc/shadow`, mas o `backup-tool` copia-o para `/tmp/leak` e eu leio a cópia. A outra via é a de **escrita**: a ferramenta escreve em qualquer sítio onde o root escreva, por isso posso pôr um ficheiro em `/root` (`pwned.txt`) ou, pior, uma chave em `/root/.ssh/authorized\_keys` ou um binário que o root execute.

(A cópia do `shadow` fica legível porque o `cp` mantém o modo da origem, `0640`, e usa o gid do meu processo. Com a host key, que é `0600`, a cópia sai ilegível — o que muda é o modo do ficheiro original, não o privilégio.)

3b. A cópia correu com **euid 0**: o `execve` pôs o effective UID a root e a ferramenta nunca o baixou. O kernel fez a verificação de permissão do `/etc/shadow` contra essa identidade efectiva e deixou passar, mesmo o real UID a ser o de `svc-report`.

O que me deu root-equivalência foi a combinação de duas coisas: o privilégio fica ligado durante toda a execução (não há altura em que o baixe) e os argumentos são meus (origem e destino à escolha). Um `cp` como root é um `cat` e um `tee` como root.

5 Exercício 4

```
$ sudo cat /etc/sudoers.d/svc-report
svc-report ALL=(ALL) NOPASSWD: /usr/bin/find
```

```
$ sudo -ln                          # como svc-report
User svc-report may run the following commands on ssc-lab:
    (ALL) NOPASSWD: /usr/bin/find

$ sudo find /etc/hostname -exec id \;
uid=0(root) gid=0(root) groups=0(root)
```

4a. O `find` tem `-exec`, que corre um comando por cada ficheiro que encontra, e como o `sudo` o lança como root, esse comando também corre como root: `-exec /bin/sh` dava-me uma shell root. A classe do erro é um comando permitido que consegue executar outros programas (*shell escape*) — o que está no GTFOBins para o `find`, o `vi`, o `tar`, o `awk`.

4b. Permitir um comando só é dar um poder limitado se esse comando não conseguir passar o privilégio a outro programa. Ou seja: não pode lançar comandos (`-exec`), não pode interpretar linguagem (shell, `awk`, `sed`), não pode ler e escrever ficheiros à escolha e não pode carregar código do ambiente (`PYTHONPATH`, `LD_PRELOAD`). Se tiver qualquer uma dessas capacidades, o direito de correr o comando passa a valer tudo o que o comando alcança.

6 Exercício 5

```
$ sudo chmod u-s /usr/local/bin/backup-tool
$ /usr/local/bin/backup-tool /etc/shadow /tmp/leak2
/usr/local/bin/backup-tool: cannot open '/etc/shadow' for reading:
Permission denied
$ /usr/local/bin/backup-tool /tmp/payload /root/pwned2.txt
/usr/local/bin/backup-tool: cannot stat '/root/pwned2.txt': Permission denied
```

5a. Tirei o bit setuid com `chmod u-s`, que é aplicar o menor privilégio: a ferramenta não mostrou precisar de root, por isso passa a correr como eu. As duas escaladas do Exercício 3 deixam de funcionar — a leitura dá `Permission denied` e a escrita também. O que perdi foi a capacidade de copiar ficheiros de outros donos, que era exactamente o excesso; copiar os ficheiros do serviço continua a dar.

5b. Medi as duas capabilities depois de tirar o setuid:

grant	cópia do `/etc/shadow`	escrita em `/root/`
`cap_dac_read_search+ep`	sucede	falha
`cap_dac_override+ep`	sucede	sucede
nenhuma (`setcap -r`)	falha	falha

Fiquei com `cap_dac_read_search`, que foi o que bastou para a leitura continuar a funcionar. A escrita ficou fechada, mas o buraco principal não: fechei só parte.

5c. Para "uma capability em vez de root" fechar o buraco e não só reduzi-lo, o poder que a ferramenta precisa de ter no trabalho normal tem de ser diferente do poder que o ataque precisa. Aqui é o mesmo: o trabalho do `cp` é ler qualquer ficheiro e escrevê-lo noutro sítio, e o ataque é ler um ficheiro que não posso ler e escrevê-lo onde não devo. A capability que o deixa trabalhar é a que me deixa atacar.

Por isso só fecham duas das três rotas: tirar o bit (5a) ou limitar os caminhos que a ferramenta

pode tocar. Tirar o bit é mais barato e mais fácil de verificar, mas a ferramenta deixa de servir para ficheiros de outros donos. Limitar os caminhos mantém a ferramenta útil, mas obriga a validar tudo em cada alteração ao código.

7 Exercício 6

```
$ sudo rm /etc/sudoers.d/svc-report
$ sudo chgrp -R svc-report /var/log/svc-report
$ sudo chmod -R g+rX,o-rwx /var/log/svc-report
$ sudo visudo -c
/etc/sudoers: parsed OK

$ cat /var/log/svc-report/run.log     # como svc-report
2026-10-28 report generated ok
$ sudo -l -U svc-report               # do lado do root
User svc-report is not allowed to run sudo on ssc-lab.
```

6a. A escalada está fechada: o `sudo -l -U` diz que o `svc-report` já não pode correr nada, e o `sudo find` com `-exec` pede password e é recusado.

O que o serviço precisava era de ler os próprios logs, e para isso não precisa de root — basta ser dono do grupo dos ficheiros. Ficou a ler os logs por permissão de grupo e nunca sai da sua identidade. Se precisasse mesmo de root, o mais seguro seria um script próprio com a lista de logs fixa, e não o `find` (é que qualquer `find` que aceite um caminho e `-exec` volta a ser uma shell root).

8 Lab 7 — capabilities: preparação

P1. Este kernel divide o root em **41** capabilities (`/proc/sys/kernel/cap_last_cap` = 40, logo bits 0 a 40); o número cresce com o kernel, e o valor de referência anda pelas quatro dezenas. Duas delas: `cap_dac_override`, que ignora as verificações de permissão de leitura, escrita e execução de ficheiros — é a que permite a um processo de root escrever em `/root` ou ler `/etc/shadow`; e `cap_net_bind_service`, que autoriza fazer *bind* a portas abaixo de 1024, a única de que o `capbind` do Lab 7 precisa.

P2. O bit setuid põe o effective UID do processo a 0 durante toda a execução, logo dá-lhe *todo* o root — todas as capabilities e os casos especiais do uid 0 — enquanto uma file capability lhe dá só a capability nomeada e deixa o euid intacto: no Lab 7, a cópia com `cap_net_bind_service+ep` ligou a :80 com `euid=1000` e `CapEff` só com esse bit, e a cópia setuid ligou com `euid=0`.

9 Lab 8 — preparação (P1, P2)

P1. No Lab 6 o `backup-tool` correu com euid 0 e copiou `/etc/shadow` para `/tmp/leak`. O DAC verificou o modo do ficheiro (`0640 root:shadow`) contra o euid e os grupos do processo: como o euid era 0, o processo era o dono e a leitura passou — o DAC não tem como recusar a root, é o que "root" significa. Uma política MAC sobre o `backup-tool` podia

tê-la recusado: um perfil que só permitisse ler os caminhos do serviço e não `/etc/shadow` teria negado o `open` mesmo com euid 0, exactamente como negou ao `lab8svc` neste lab. E também teria negado a escrita em `/root`. O MAC não substitui o DAC: acrescenta uma verificação depois dele, que o root não contorna.

P2. Um Gitea que eu administro: toca em repositórios, LFS, avatares, temporários e ainda lança `git` e `ssh` para hooks e espelhos, portanto tem muitos caminhos e muitos execs, e um perfil demasiado apertado tira-o do ar. Carregava-o primeiro em complain para ver o que ele pede em funcionamento normal e só depois passava a enforce. O risco que aceito é que, durante essa janela, o perfil não impede nada: se o Gitea for comprometido enquanto está em complain, o acesso acontece e só fica registado — e o ruído de muitos `ALLOWED` pode esconder a tentativa que interessava.

10 Lab 8 — AppArmor: o perfil mínimo (2a)

O alvo é uma cópia do `cat`, o "leitor de relatórios" cujo trabalho é ler um ficheiro, para que as leituras permitidas e recusadas sejam fáceis de observar: `sudo cp /bin/cat /usr/local/bin/lab8svc`. O perfil foi escrito à mão, para se ver cada regra, e carregado em enforce:

```
#include <tunables/global>
/usr/local/bin/lab8svc {
  #include <abstractions/base>
  /usr/local/bin/lab8svc mr,
  /etc/hostname r,
}
```

Duas regras explícitas — o binário pode mapear-se e ler-se (`mr`) e pode ler `/etc/hostname` (`r`) — e o `#include <abstractions/base>`, que é política e não comentário: acrescenta o que qualquer programa precisa para arrancar (bibliotecas do loader, locale, `/dev/urandom`, escrita em `/dev/null` e no socket de log, parte de `/proc`). Carregado com `apparmor_parser -r`, aparece como `/usr/local/bin/lab8svc (enforce)`.

Previsão, antes de correr: `/etc/hostname` passa, porque tem regra `r`; `/etc/shadow` é recusado, porque não tem regra nenhuma e o perfil está em enforce. Correr com `sudo` não muda nada: o DAC deixa passar (root), o MAC não.

```
$ sudo /usr/local/bin/lab8svc /etc/hostname
ssc-lab
exit=0

$ sudo /usr/local/bin/lab8svc /etc/shadow
/usr/local/bin/lab8svc: /etc/shadow: Permission denied
exit=1
```

Bateu certo com a previsão. Outro ficheiro que o `cat` como root lê e o perfil recusa: `/etc/passwd`.

```
$ sudo cat /etc/passwd >/dev/null && echo "(root cat: OK)"
(root cat: OK)
$ sudo /usr/local/bin/lab8svc /etc/passwd
/usr/local/bin/lab8svc: /etc/passwd: Permission denied
exit=1
```

`abstractions/base` concede estas entradas de `/proc`:

- `@{PROC}/sys/kernel/version`
- `@{PROC}/sys/kernel/ngroups_max`
- `@{PROC}/sys/kernel/cap_last_cap`
- `@{PROC}/meminfo`, `@{PROC}/stat`, `@{PROC}/cpuinfo`
- `@{PROC}/filesystems`
- `@{PROC}/sys/vm/overcommit_memory`
- `@{PROC}/@{pid}/{maps,auxv,status}`

Como `@{pid}` é o padrão dos PIDs, `/proc/self/status` cai nesta última regra. Aberto: `/proc/self/status`; fechado: `/proc/self/mounts`.

```
$ sudo /usr/local/bin/lab8svc /proc/self/status | head -2
Name: lab8svc
Umask:  0022
exit=0

$ sudo cat /proc/self/mounts >/dev/null && echo "(root cat: OK)"
(root cat: OK)
$ sudo /usr/local/bin/lab8svc /proc/self/mounts
/usr/local/bin/lab8svc: /proc/self/mounts: Permission denied
exit=1
```

Os resultados batem certo com a lista: `status` está no `abstractions/base`, `mounts` não. É a diferença entre o que se pensa que o programa toca e o que a política permite — precisamente o que um atacante procura.

11 Lab 8 — MAC contra DAC e least privilege (3a, 3b)

A negação da leitura de `/etc/shadow`, no audit log:

```
type=AVC msg=audit(1794399687.541:3250): apparmor="DENIED" operation="open"
  class="file" profile="/usr/local/bin/lab8svc" name="/etc/shadow" pid=17879
  comm="lab8svc" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
  FSUID="root" OUID="root"
```

3a. O processo de root passou todas as verificações DAC — corre com `fsuid=0`, como a linha mostra — e foi recusado à mesma. Um perfil AppArmor pode proibir, por caminho, abrir, ler, escrever, criar ou mapear ficheiros, usar capabilities, abrir sockets, fazer mount, fazer ptrace e executar outros programas, independentemente do UID. Aqui proibiu `open` de `/etc/shadow` a um processo que, no DAC, tinha todo o direito de o abrir.

Isso faz do MAC um controlo de outra natureza, e não apenas permissões de ficheiro mais fortes: no DAC o dono do objecto decide, e o root pode sempre ignorar; no MAC a política é do sistema, está ligada ao *programa* e não à identidade, e nem o root nem o dono do ficheiro a podem contornar. A pergunta deixa de ser "quem é o utilizador?" e passa a ser "que programa é este e o que lhe foi permitido?".

O que o DAC faz bem e o MAC não substitui é a propriedade e a partilha de dados: `chmod`, `chown` e ACLs dizem quem é o dono de um ficheiro e com quem se partilha, sem exigir uma política por programa. O MAC não exprime "este ficheiro é do hugo" nem resolve partilha;

precisa de um perfil por binário e de o manter.

Para distinguir um ficheiro de que o serviço precisa de um a que o atacante o obrigou a chegar: primeiro a linha de base do comportamento normal (o que ele abre a fazer o seu trabalho, medido em complain); depois, a relação com a função e a configuração do serviço; e, por fim, a origem do caminho — se vem de input do utilizador, se é uma travessia `../`, e em que sequência aparece. Um leitor de relatórios que abre `/etc/shadow` a seguir a um pedido externo não "precisa" dele: foi levado lá.

O passo de least privilege

Removida a regra `/etc/hostname r,` e recarregado o perfil, a leitura que antes passava passou a ser recusada. O perfil final tem só o `abstractions/base` e a regra do próprio binário:

```
#include <tunables/global>
/usr/local/bin/lab8svc {
  #include <abstractions/base>
  /usr/local/bin/lab8svc mr,
}
```

```
$ sudo /usr/local/bin/lab8svc /etc/hostname
/usr/local/bin/lab8svc: /etc/hostname: Permission denied
exit=1
```

Removi a regra porque o trabalho de um leitor de relatórios não inclui `/etc/hostname`: a regra existia para a demonstração, e cada caminho permitido a mais é superfície de ataque.

3b. O perfil final impede o atacante de **executar outro programa**: não tem nenhuma regra de execução, portanto quem tomasse o serviço não conseguia lançar uma shell, um `cat` ou um `curl` para a segunda fase. Impede também a leitura de tudo o que está fora do `abstractions/base` (`/etc/shadow`, `/etc/passwd`, chaves SSH, `/proc/self/mounts`) e a escrita de ficheiros — só `/dev/null` e o socket de log vêm do base. A shell é a que ele quer mais: sem exec, o compromisso fica preso ao binário.

12 Lab 8 — a negação como sinal de detecção (4a, 4b)

Feita mais uma leitura proibida, procurei-a como um defensor a procuraria. As duas ferramentas de primeira linha falharam: `ausearch -m AVC -ts recent` devolveu `<no matches>` (procura AVCs no formato do SELinux e os registos do AppArmor não têm `tclass=`), e `dmesg | grep -i apparmor` só mostrou linhas `apparmor="STATUS"` do arranque, porque com o auditd a correr o kernel entrega os registos ao auditd. A negação estava no ficheiro:

```
$ sudo grep -a 'apparmor="DENIED"' /var/log/audit/audit.log | grep -a shadow |
    tail
type=AVC msg=audit(1794399868.918:3547): apparmor="DENIED" operation="open"
  class="file" profile="/usr/local/bin/lab8svc" name="/etc/shadow" pid=18326
  comm="lab8svc" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
  FSUID="root" OUID="root"
```

4a — nota de defensor

campo	resposta
Evidência (linha e valores de `apparmor`, `operation`, `profile`, `name`, `comm`, `pid`)	`apparmor="DENIED"`, `operation="open"`, `profile="/usr/local/bin/lab8svc"`, `name="/etc/shadow"`, `comm="lab8svc"`, `pid=18326`
O que foi tentado	O processo `lab8svc` (pid 18326, a correr com `fsuid=0`, root) tentou abrir `/etc/shadow` para leitura (`requested_mask="r"`).
O que o travou	O perfil `/usr/local/bin/lab8svc`, em **enforce** (confirmado em `/sys/kernel/security/apparmor/profiles`).
Como o encontrei	`ausearch -m AVC -ts recent` não deu nada; `dmesg | grep -i apparmor` só tinha STATUS do arranque; funcionou `grep -a 'apparmor="DENIED"'` no `/var/log/audit/audit.log`.
O que a linha não diz	Porquê nem quem o mandou fazer: não tem linha de comando, processo pai, sessão do utilizador, nem o pedido externo que levou a isto; não diz se era ataque ou erro de configuração, nem o que aconteceu antes ou depois, nem quantas tentativas houve noutros caminhos.

Antes de um incidente, para verificar que a ferramenta em que penso confiar mostra mesmo este tipo de evento, provocava uma negação conhecida — correr o binário confinado contra um ficheiro que o perfil não permite — e confirmava que a ferramenta mostra essa linha com o evento garantido, e que uma resposta vazia quer dizer "não houve evento" e não "a ferramenta não vê isto". Com o `ausearch` teria descoberto logo que ele não vê AppArmor.

4b. Em **enforce** ganho a prevenção — a leitura foi parada, o dado não saiu — e perco a observação do que o atacante faria a seguir; o log diz `DENIED` e nada mais. Em **complain** ganho a observação completa (o acesso aconteceu e ficou registado como `ALLOWED`, com o `denied_mask` que teria sido aplicado) e perco o controlo: o dado foi lido, o dano é real.

São precisos os dois porque são metades diferentes do mesmo trabalho: o controlo que bloqueia impede o dano mas não me diz que estou a ser atacado; o log diz-me que estou a ser atacado mas, sem o controlo, só depois do dano. Um SIEM ou uma pessoa que lê o log precisa do registo; a política precisa de bloquear. Além disso, os `DENIED` são o que permite afinar o perfil e o que serve de evidência depois — e um ataque que falha também é informação: diz que alguém está a tentar.

13 Lab 8 — a vista cross-platform (5)

Os dois casos medidos — o `lab8svc` impedido de ler `/etc/shadow` e a regra para `/bin/cat` que deixou de casar — vistos por quatro mecanismos equivalentes:

- **SELinux (type enforcement, por label).** Cada processo corre num domínio e cada ficheiro tem um tipo; a política diz que o domínio do `lab8svc` não pode ler `shadow_t`. A leitura é negada com `EACCES` e registada como AVC — com o `tclass=file` que o `ausearch` procura e que os registos do AppArmor não têm. Quem escreve a restrição é o administrador/distribuição, em módulos de política compilados; o processo não muda o seu label. O problema do 3d-ii não acontece: o controlo é sobre o inode, por isso `/bin/cat` e `/usr/bin/cat`, sendo o mesmo ficheiro, têm o mesmo label e o nome usado para lá chegar é irrelevante. O que falha é um label errado, não um caminho.
- **FreeBSD jails (namespace).** O jail dá ao processo uma vista do sistema de ficheiros: só existe a subárvore que o administrador montou como raiz do jail, e o `/etc/shadow` do host não lá está. Se o jail tiver o seu próprio `/etc/shadow`, um root dentro do jail pode lê-lo: o jail

limita o que é visível, não o que root faz dentro do visível. Quem escreve é o administrador (configuração do jail, mounts, parâmetros `allow.*`). O 3d-ii não se aplica da mesma maneira: não há regras por caminho a casar, e um symlink dentro do jail resolve-se dentro da raiz do jail — o risco está num mount que exponha o que não devia.

- **OpenBSD pledge/unveil (auto-restrição).** O próprio programa declara as paths que quer ver e as classes de syscalls que quer usar: um `cat` com `unveil("/etc/hostname","r")` e `pledge("stdio rpath")` não abre `/etc/shadow` — a path não foi unveilada e o kernel responde `ENOENT`/`EACCES` — e as syscalls fora da promessa são bloqueadas. Quem escreve é o programador, não o administrador. O problema do 3d-ii pode acontecer, porque unveil também é por nome: mas é o programa que unveila o caminho que vai abrir e depois fecha a lista (`unveil(NULL,NULL)`), pelo que o erro é do código e não de um parser silencioso.
- **Windows (integrity levels e tokens).** O token do processo carrega o integrity level (low/medium/high/system) e os privilégios, e cada objecto tem um security descriptor e um label de integridade. O SAM só é acessível a SYSTEM pela DACL, por isso um processo medium não o abre; além disso o MIC impõe *no-write-up* (um low não escreve num medium/high), embora a leitura para cima seja em geral permitida. Quem escreve é o administrador/SO (ACLs e labels), com o manifesto da aplicação a declarar capabilities. O 3d-ii não acontece: a decisão é sobre o objecto (descriptor), não sobre o nome — os dois caminhos do `cat` seriam o mesmo objecto com o mesmo descriptor.

A ideia comum. Os cinco — AppArmor incluído — tiram a decisão de "quem é o utilizador" e põem-na numa política que o processo não pode reescrever: o perfil do programa, o label do inode, a vista do namespace, as promessas que o programa se impôs, ou o token e o integrity level. É o monitor de referência: mediação completa, à prova de manipulação pelo sujeito, e pequeno o suficiente para se poder verificar.

O que o modelo por caminho do AppArmor torna fácil e frágil. Fácil: escreve-se na linguagem em que o administrador pensa (caminhos), por programa, sem relabelar o sistema e sem tocar no código; um perfil de vinte linhas lê-se todo. E foi por isso que aqui se viu logo o que estava permitido e o que não. Frágil: o caminho é a identidade do objecto, e o mesmo ficheiro tem vários nomes — symlinks, hardlinks, bind mounts, renomeações, chroot e namespaces de mount. A regra para `/bin/cat` não casou com `/usr/bin/cat` e o parser não disse nada; o mesmo pode acontecer com `/bin/sh` ou com um caminho que muda depois de o perfil ser escrito. O SELinux evita isto ao colar o label ao inode uma vez; o AppArmor paga a legibilidade com essa manutenção.

A Reflexão — Lab 6: setuid e sudo

Setuid e sudo (200–300 palavras)

Os dois mecanismos resolvem o mesmo problema de maneira diferente: o setuid dá o privilégio ao *binário*, o sudo dá-o por *política*. O setuid é local e não se vê — o privilégio está no bit do ficheiro e só o encontro com `find -perm -4000`. O sudo é escrito e lista-se com `sudo -l`, mas o que ele dá é o poder do programa permitido, não o que a regra queria dizer: o `find` com `-exec` é uma shell root disfarçada de pesquisa de logs.

Achei o setuid mais fácil de pensar, porque a lista de binários é curta e não muda. O sudo foi mais fácil de explorar: não tive de descobrir nada, bastou ler o `/etc/sudoers.d` e ver o que o `find` sabe fazer.

Nos dois casos o kernel não tinha nenhum bug — o problema foi usar bem duas coisas mal configuradas, como no Lab 1 com o script do cron e no Lab 5 com o `ForwardAgent`. Acho

que o operador deve olhar para o que corre, em nome de quem, e para o que esse programa consegue fazer; ao rever o sudoers a pergunta não é "que comando é este" mas "o que é que este comando pode executar".

No capstone começava pela regra de sudo, porque é uma linha, não obriga a mexer em binários e fecha logo a via mais fácil. O setuid ficava para depois.

B Reflexão — Lab 7: capabilities

Capabilities (200–300 palavras)

O setuid continua em todo o lado porque custa um bit: `chmod u+s` e o programa de root clássico funciona, sem saber nada de capabilities. As capabilities pedem mais — o binário tem de ser revisto para declarar só o que precisa, o `setcap` passa a fazer parte do empacotamento e do build, e o `+e` mal posto faz o `execve` falhar com EPERM (Ex. 6). Pior, um programa antigo que baixa privilégios com `setuid(getuid())` não foi escrito para viver com capabilities, e o que era um `chmod` passa a ser uma decisão de arquitectura. A ideia é simples; a adopção é que não é.

Vi duas maneiras de a restrição cair: o `--privileged` devolve as 41 capabilities, e o sysctl do Docker põe a porta 80 fora do gate. A generalização é que a capability é uma condição, não a política. O kernel decide com o `CapEff`, mas também com o `CapBnd`, com as file capabilities da imagem, com o sysctl, com o seccomp e com os namespaces. "Tirei a capability perigosa" fala de uma peça; a acção perigosa pode continuar possível por outra via — a porta deixa de ser privilegiada, o device aparece no `/dev`, o binário traz a capability. É a lição do Lab 6 outra vez: o mecanismo não é a garantia.

No capstone, das capabilities que nomeei no Ex. 3a, começava por `cap_sys_admin` e tirava-a primeiro: é a que transforma "root dentro do contentor" em root do host, porque permite montar o disco do host e manipular cgroups e namespaces. Verificava lendo os conjuntos dentro do contentor (`grep Cap /proc/self/status`) e descodificando com `capsh --decode`: o bit 21 não pode estar no `CapEff` nem no `CapBnd`, e a prova final é tentar o `mount` e vê-lo falhar.

C Reflexão — Lab 8: AppArmor

AppArmor e o sinal de detecção (200–300 palavras)

A negação é o mesmo registo que usei para afinar o perfil e para detectar o ataque. Se aplicar o perfil em enforce e nunca ler o log, ganho a prevenção e perco tudo o resto: não sei que estou a ser sondado nem tenho evidência para responder. O ataque fica invisível precisamente por ter falhado, e quem insiste noutro caminho continua a não ser visto. O log é o que transforma bloqueios isolados numa série de tentativas.

O modelo por caminho falha quando o caminho não é a identidade: symlinks, hardlinks, bind mounts e renames fazem o mesmo ficheiro aparecer com outro nome, e a regra deixa de casar — foi o que aconteceu com `/bin/cat` e `/usr/bin/cat`, sem o parser dizer nada. Um atacante pode criar um caminho novo para um alvo permitido, e a regra escrita para o nome antigo não dispara. O SELinux cola o label ao inode: o nome não muda a decisão e um ficheiro movido mantém o tipo; o custo é manter labels e relabelar.

Capstone, um passo que não o 4a: a leitura de `/etc/shadow` em complain (3c). O `lab8svc` a correr como root abre e lê o ficheiro (`exit=0`); o registo é um AVC `apparmor="ALLOWED"`, `operation="open"`, com perfil, `name`, `pid`, `comm`, `fsuid=0` e `denied_mask="r"`. Só com esse registo, um defensor distingue-o de uma acção legítima? Não: diz que o leitor abriu o shadow, o que é suspeito, mas é igual quer venha de um atacante quer de um trabalho mal configurado — falta actor, linha de comando e intenção. Só correlacionando com a linha de base e com o pedido anterior.